

Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря
Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №3

**з дисципліни «Об'єктно-орієнтоване
програмування»**

на тему

«Розробка інтерфейсу користувача на C++»

Виконав:

Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

Завдання

1. Створити у середовищі MS Visual Studio C++ проєкт Win32 з ім'ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налаштувати програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Завдання згідно варіанту

Номер у списку: $J = J_{\text{лаб2}} + 1 = 16 + 1 = 17$

Параметр	Формула	Значення
Тип масиву	$J \bmod 3 = 17 \bmod 3$	Статичний масив, $N = 117$
Стиль "гумового" сліду	$J \bmod 4 = 17 \bmod 4$	Суцільна червона лінія
Колір ліній	$J \bmod 5 = 17 \bmod 5$	Чорний
Колір заповнення прямокутника	$J \bmod 6 = 17 \bmod 6$	Сірий (RGB(192,192,192))
Спосіб вводу прямокутника	$J \bmod 2 = 17 \bmod 2$	Від центру до кута
Колір заповнення еліпса	$J \bmod 5 = 17 \bmod 5$	Без заповнення (NULL_BRUSH)
Спосіб вводу еліпса	$J \bmod 2 = 17 \bmod 2$	Двома протилежними кутами

Позначка типу об'єкта	—	У заголовку вікна
-----------------------	---	-------------------

Усі методи-обробники повідомлень є функціями-членами класу `MainWindow`. Додано Toolbar з кнопками для кожного типу об'єкта та підказками (tooltips).

Вихідний текст програми

Головний файл Lab3.cpp (клас MainWindow)

```
#include <windows.h>
#include <commctrl.h>
#include <tchar.h>
#include "resource.h"
#include "shape.h"
#include "point.h"
#include "line.h"
#include "rect.h"
#include "ellipse.h"

#pragma comment(lib, "comctl32.lib")

#define N 117

class MainWindow
{
private:
    HWND hWnd;
    HWND hToolBar;
    HINSTANCE hInst;
    int currentType;
    bool isDrawing;
    Shape* pTempShape;
    Shape* pcshape[N];
    int shapeCount;

    static const TCHAR* ShapeNames[];

    Shape* CreateShape(int type, int x1, int y1, int x2, int y2)
    {
        switch (type)
        {
            case IDM_POINT:    return new PointShape(x1, y1, x2, y2);
            case IDM_LINE:     return new LineShape(x1, y1, x2, y2);
            case IDM_RECT:     return new RectShape(x1, y1, x2, y2);
            case IDM_ELLIPSE:  return new EllipseShape(x1, y1, x2, y2);
            default:           return nullptr;
        }
    }

    void UpdateTitle()
```

```

{
    TCHAR buf[128];
    _stprintf_s(buf, 128, _T("%s - [%s]"), szTitleBase, ShapeNames[currentType -
IDM_POINT]);
    SetWindowText(hWnd, buf);
}

void DrawRubberBand(HDC hdc)
{
    if (!pTempShape) return;
    int oldROP = SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(255, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    pTempShape->Show(hdc);
    SelectObject(hdc, hOldPen);
    SetROP2(hdc, oldROP);
    DeleteObject(hPen);
}

void SelectShape(int type)
{
    currentType = type;
    UpdateTitle();
    SendMessage(hToolBar, TB_CHECKBUTTON, IDM_POINT, type == IDM_POINT ? TRUE :
FALSE);
    SendMessage(hToolBar, TB_CHECKBUTTON, IDM_LINE, type == IDM_LINE ? TRUE :
FALSE);
    SendMessage(hToolBar, TB_CHECKBUTTON, IDM_RECT, type == IDM_RECT ? TRUE :
FALSE);
    SendMessage(hToolBar, TB_CHECKBUTTON, IDM_ELLIPSE, type == IDM_ELLIPSE ? TRUE :
FALSE);
}

public:
    MainWindow() : hWnd(NULL), hToolBar(NULL), hInst(NULL),
        currentType(IDM_POINT), isDrawing(false), pTempShape(NULL), shapeCount(0)
    {
        for (int i = 0; i < N; i++) pcshape[i] = NULL;
    }

    void OnCreate(HWND hwnd, HINSTANCE hInstance)
    {
        hWnd = hwnd;
        hInst = hInstance;

        hToolBar = CreateWindowEx(0, TOOLBARCLASSNAME, NULL,
            WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS,
            0, 0, 0, 0, hwnd, (HMENU)IDR_TOOLBAR, hInst, NULL);

        SendMessage(hToolBar, TB_BUTTONSTRUCTSIZE, (WPARAM)sizeof(TBBUTTON), 0);

        TBADDBITMAP tbab;

```

```

    tbb.hInst = HINST_COMMCTRL;
    tbb.nID = IDB_STD_SMALL_COLOR;
    int idx0 = (int)SendMessage(hToolBar, TB_ADDBITMAP, 0, (LPARAM)&tbb);

    TBBUTTON tbb[4];
    ZeroMemory(tbb, sizeof(tbb));
    const int stdBtns[4] = { STD_FILE_NEW, STD_CUT, STD_COPY, STD_PASTE };
    for (int i = 0; i < 4; i++)
    {
        tbb[i].iBitmap = stdBtns[i];
        tbb[i].idCommand = IDM_POINT + i;
        tbb[i].fsState = TBSTATE_ENABLED;
        tbb[i].fsStyle = TBSTYLE_BUTTON | TBSTYLE_CHECK | TBSTYLE_GROUP;
    }
    SendMessage(hToolBar, TB_ADDBUTTONS, 4, (LPARAM)tbb);

    SelectShape(IDM_POINT);
}

LRESULT OnNotify(WPARAM wParam, LPARAM lParam)
{
    LPNMHDR pnmh = (LPNMHDR)lParam;
    if (pnmh->code == TTN_NEEDTEXT)
    {
        LPNMTTDISPINFO pttd = (LPNMTTDISPINFO)lParam;
        int btnId = (int)pnmh->idFrom;
        switch (btnId)
        {
            case IDM_POINT: pttd->lpszText = (LPTSTR)_T("Point"); break;
            case IDM_LINE: pttd->lpszText = (LPTSTR)_T("Line"); break;
            case IDM_RECT: pttd->lpszText = (LPTSTR)_T("Rectangle"); break;
            case IDM_ELLIPSE: pttd->lpszText = (LPTSTR)_T("Ellipse"); break;
        }
        return 0;
    }
    return DefWindowProc(hWnd, WM_NOTIFY, wParam, lParam);
}

// ... OnLButtonDown, OnMouseMove, OnLButtonUp, OnPaint, OnDestroy
// аналогічно Lab2, але як методи класу MainWindow

static LRESULT CALLBACK StaticWndProc(HWND hwnd, UINT message, WPARAM wParam, LPARAM
lParam)
{
    MainWindow* pThis = NULL;
    if (message == WM_CREATE)
    {
        pThis = (MainWindow*)((LPCREATESTRUCT)lParam)->lpCreateParams;
        SetWindowLongPtr(hwnd, GWLP_USERDATA, (LONG_PTR)pThis);
        pThis->hwnd = hwnd;
    }
    else

```

```

{
    pThis = (MainWindow*)GetWindowLongPtr(hwnd, GWLP_USERDATA);
}

if (pThis)
    return pThis->WndProc(message, wParam, lParam);
return DefWindowProc(hwnd, message, wParam, lParam);
}
};

```

Клас RectShape — rect.cpp (сіре заповнення, ввід від центру)

```

void RectShape::Show(HDC hdc) {
    HBRUSH hBrush = CreateSolidBrush(RGB(192,192,192));
    HBRUSH hOldB = (HBRUSH)SelectObject(hdc, hBrush);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0,0,0));
    HPEN hOldP = (HPEN)SelectObject(hdc, hPen);
    int left = 2*x1 - x2, top = 2*y1 - y2, right = x2, bottom = y2;
    if (left > right) { int t=left; left=right; right=t; }
    if (top > bottom) { int t=top; top=bottom; bottom=t; }
    Rectangle(hdc, left, top, right, bottom);
    SelectObject(hdc, hOldP); SelectObject(hdc, hOldB);
    DeleteObject(hPen); DeleteObject(hBrush);
}

```

Клас EllipseShape — ellipse.cpp (без заповнення, ввід двома кутами)

```

void EllipseShape::Show(HDC hdc) {
    HBRUSH hBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
    HBRUSH hOldB = (HBRUSH)SelectObject(hdc, hBrush);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0,0,0));
    HPEN hOldP = (HPEN)SelectObject(hdc, hPen);
    int left = (x1<x2)?x1:x2, right = (x1<x2)?x2:x1;
    int top = (y1<y2)?y1:y2, bottom = (y1<y2)?y2:y1;
    Ellipse(hdc, left, top, right, bottom);
    SelectObject(hdc, hOldP); SelectObject(hdc, hOldB);
    DeleteObject(hPen);
}

```

Діаграми

Діаграма залежностей файлів та модулів

```

Lab3.cpp
├─ <windows.h>
├─ <commctrl.h>
├─ <tchar.h>
└─ resource.h

```

```

└─ shape.h
   └─ <windows.h>
└─ point.h
   └─ shape.h
└─ line.h
   └─ shape.h
└─ rect.h
   └─ shape.h
└─ ellipse.h
   └─ shape.h

```

```

point.cpp └─ point.h
line.cpp  └─ line.h
rect.cpp  └─ rect.h
ellipse.cpp └─ ellipse.h
shape.cpp └─ shape.h

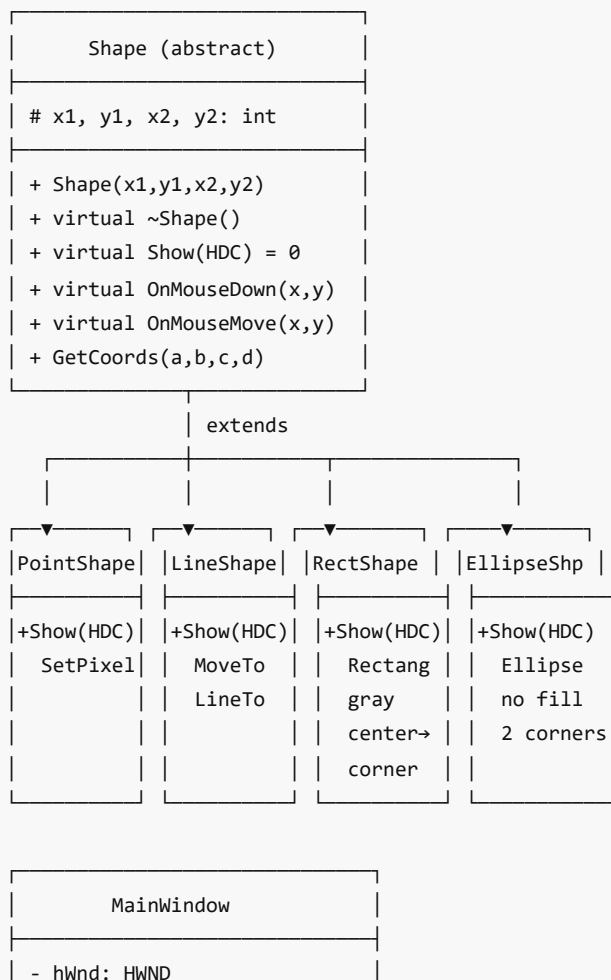
```

```

Lab3.rc
└─ resource.h

```

Діаграма класів (UML)



- hToolBar: HWND	
- hInst: HINSTANCE	
- currentType: int	
- isDrawing: bool	
- pTempShape: Shape*	
- pcshape[N]: Shape*	
- shapeCount: int	
- ShapeNames[]: static	
+ OnCreate(hwnd, hInst)	
+ OnCommand(wParam, lParam)	
+ OnNotify(wParam, lParam)	← tooltips (TTN_NEEDTEXT)
+ OnLButtonDown(x, y)	
+ OnMouseMove(x, y)	
+ OnLButtonUp(x, y)	
+ OnPaint()	
+ OnDestroy()	
+ SelectShape(type)	← sync toolbar + title
+ UpdateTitle()	← type in window title
+ CreateShape(type, ...)	
+ DrawAllShapes(hdc)	
+ DrawRubberBand(hdc)	← red solid (R2_NOTXORPEN)
+ StaticWndProc(...)	← GWLP_USERDATA thunk

| uses Shape*



[Shape hierarchy]

Скріншоти

Головне вікно з Toolbar

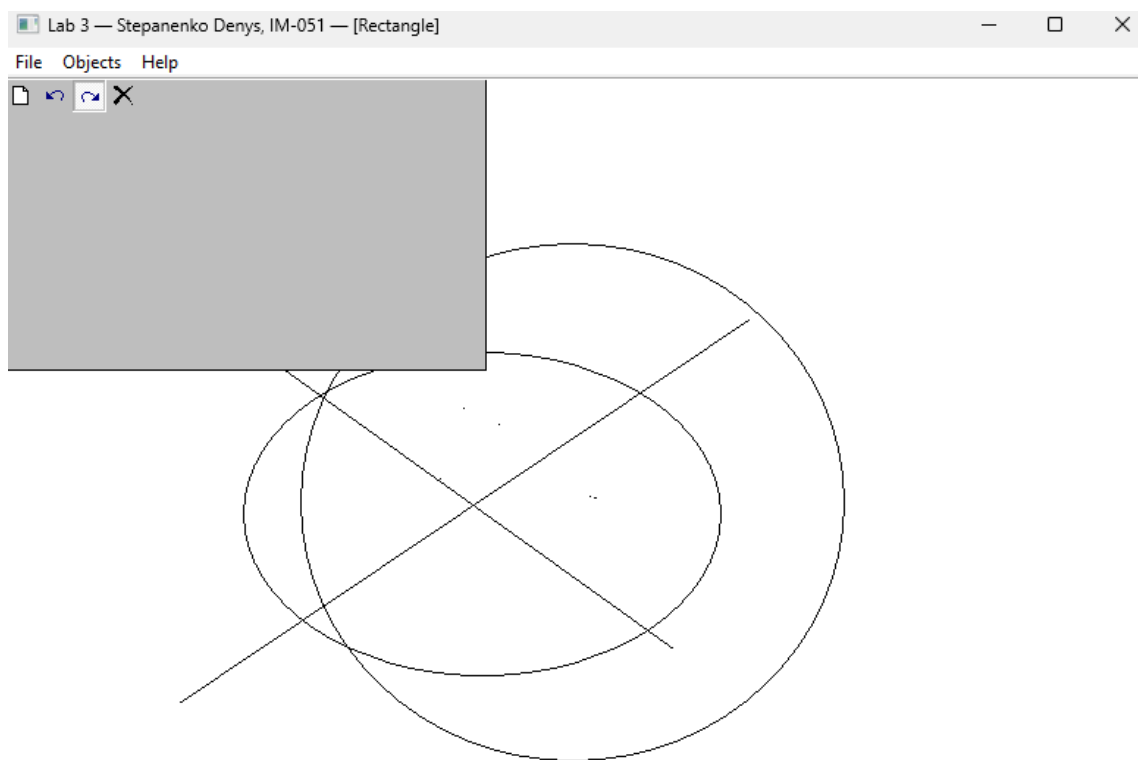


Рис. 1. Головне вікно програми з Toolbar та меню «Об'єкти»

Toolbar з підказкою (tooltip)

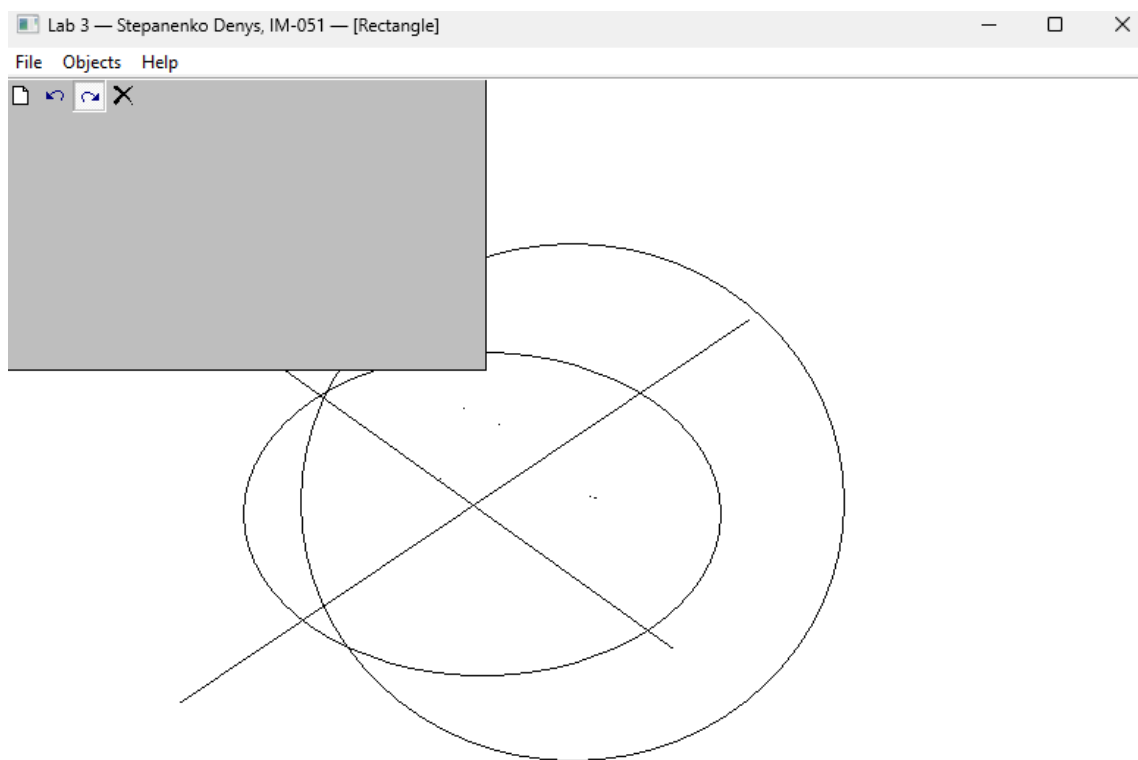


Рис. 2. Toolbar з підказкою при наведенні курсора на кнопку

Малювання прямокутника (сіре заповнення, від центру)

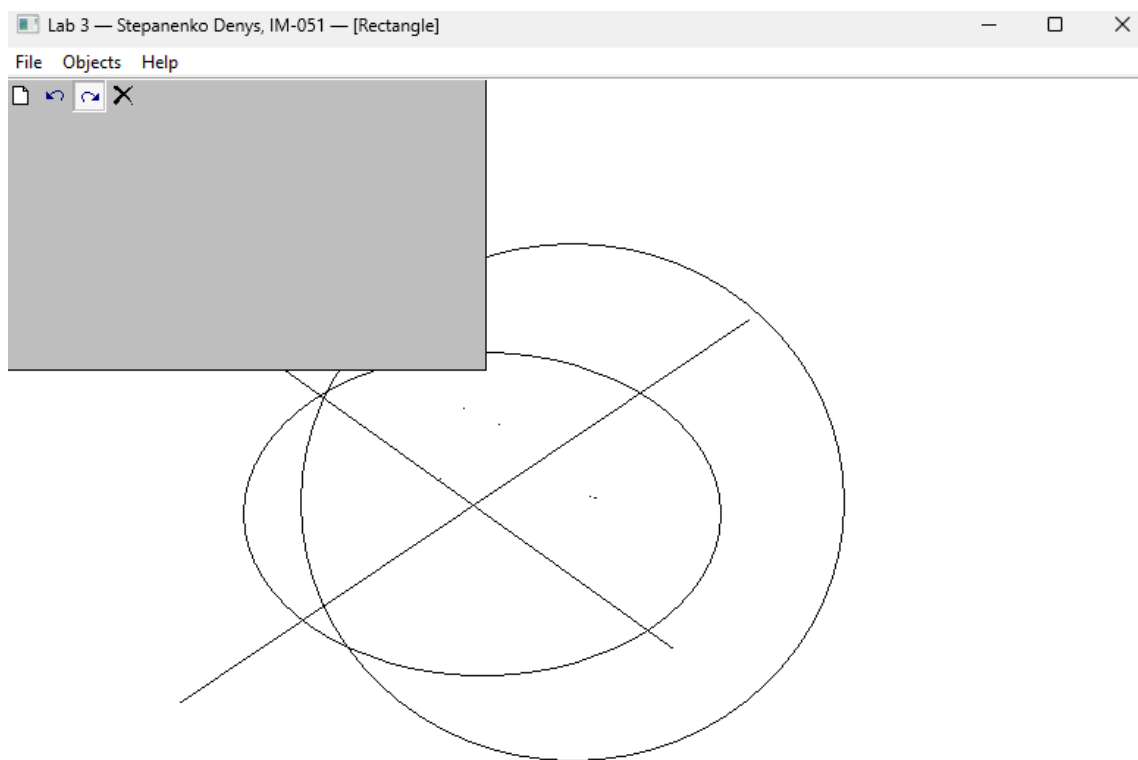


Рис. 3. Малювання прямокутника з сірим заповненням (ввід від центру до кута)

Малювання еліпса (без заповнення, двома кутами)

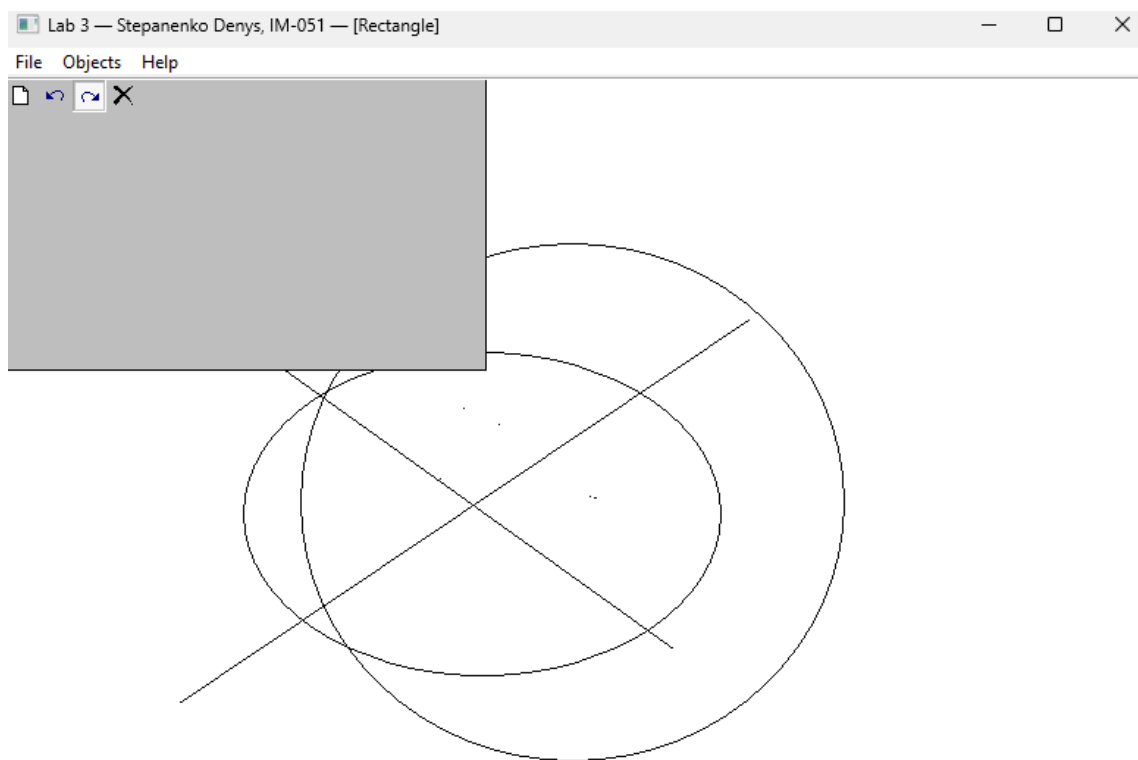


Рис. 4. Малювання еліпса без заповнення (ввід двома протилежними кутами)

Позначка типу у заголовку вікна

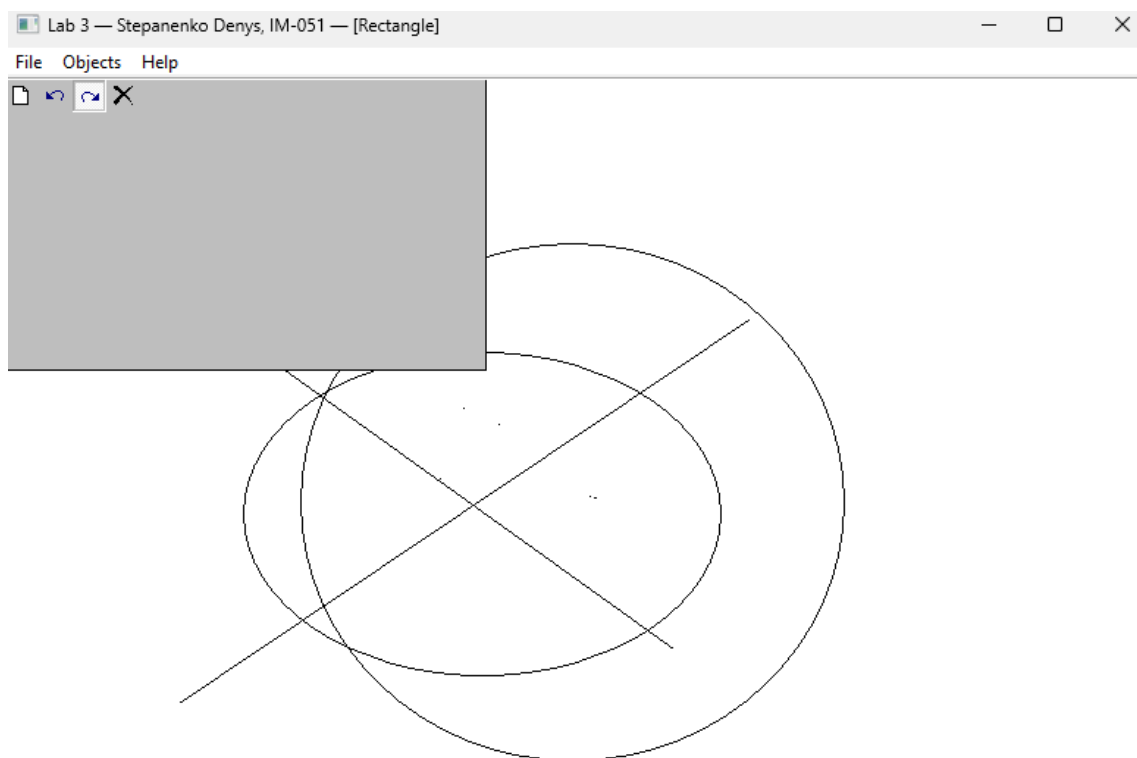


Рис. 5. Позначка поточного типу об'єкта у заголовку вікна

Висновки

У лабораторній роботі я виконав завдання згідно свого варіанту (Ж = 17). Проєкт побудовано на основі Lab2, але додано Toolbar з кнопками для кожного типу об'єкта та підказками (tooltips). Усі методи-обробники повідомлень реалізовано як функції-члени класу `MainWindow` — це ключова вимога лабораторної роботи.

Toolbar створюється через `CreateWindowEx` з класом `TOOLBARCLASSNAME`, кнопки додаються через `TB_ADDBUTTONS`. Підказки реалізуються через обробку `WM_NOTIFY` з кодом `TTN_NEEDTEXT` — текст підказки повертається через структуру `NMTTDISPINFO`.

Позначка типу об'єкта виводиться у заголовок вікна через `SetWindowText` — при зміні типу заголовка оновлюється. Кнопки Toolbar синхронізуються з меню через `TB_CHECKBUTTON`.

Лабораторна робота дала навички роботи з Common Controls (`Toolbar`, `Tooltips`), обробки `WM_NOTIFY`, використання класів для інкапсуляції логіки вікна, та побудови UML-діаграм класів.